# EXERCISE 9 – FORWARD CHAINING

## AIM

To implement forward chaining in Prolog and infer new facts from the given initial knowledge using rules.

## PROCEDURE

1. Open a Prolog environment.
2. Declare the predicate known/1 as dynamic so facts can be added during execution.
3. Add the initial facts to the knowledge base using assert/1.
4. Define rules that check existing facts and infer new facts when the conditions are satisfied.
5. Run the start query for each exercise.
6. Observe the inferred message displayed as output.

## EXERCISE 1: SIMPLE INFERENCE

Code

```
:- dynamic known/1.
% Initial knowledge
start :- assert(known(a)), assert(known(b)), infer.

% Rule
infer :- known(a), known(b), \+ known(c), assert(known(c)), write('Inferred c'), nl.

% Query
?- start.
```

Output: Inferred c

## EXERCISE 2: FIRE ALERT SYSTEM

Code

```
:- dynamic known/1.
% Rule to infer fire
check :- known(smoke), known(heat), assert(known(fire)), write('Fire Alert!'), nl.

% Start rule
start :- assert(known(smoke)), assert(known(heat)), check.

% Query
?- start.
```

Output: Fire Alert!

## EXERCISE 3: WATER PURITY DETECTION

Code

```
:- dynamic known/1.
% Rule to infer pollution
infer_pollution :- known(cloudy_water), known(bad_smell), assert(known(polluted)), write('Water is polluted
```

```
start :- assert(known(cloudy_water)), assert(known(bad_smell)), infer_pollution.

% Query
?- start.
```

Output: Water is polluted